

Security tradeoffs

Security provides confidentiality, integrity, and availability assurances for a workload's systems and for its users' data. Security controls are required across the workload and across the software development and operational components that support it. Workload teams can almost never compromise on security controls.

When you design a workload, consider how decisions based on the [Security design principles](#) and the recommendations in the [Design review checklist for Security](#) might influence the goals and optimizations of other pillars. Some security decisions benefit one pillar but constitute tradeoffs for another. This article describes example tradeoffs that a workload team might encounter when establishing security assurances.

Security tradeoffs with Reliability



Tradeoff: Increased complexity. The Reliability pillar prioritizes simplicity and recommends that points of failure are minimized.

- Some security controls can increase the risk of misconfiguration, which can lead to service disruption. Examples of security controls that can introduce misconfiguration include network traffic rules, identity providers, virus scanning exclusions, and role-based or attribute-based access control assignments.
- Increased segmentation usually results in a more complex environment in terms of resource and network topology and operator access. This complexity can lead to more points of failure in processes and in workload execution.
- Workload security tooling spans many layers of a workload's architecture, operations, and runtime requirements. These tools can affect resiliency, availability, and capacity planning. Failing to account for tooling limitations can cause reliability events, like SNAT port exhaustion on an egress firewall.



Tradeoff: Increased critical dependencies. The Reliability pillar recommends minimizing critical dependencies. A workload that minimizes critical dependencies, especially external ones, has more control over its points of failure.

The Security pillar requires a workload to explicitly verify identities and actions. Verification occurs via critical dependencies on key security components. If those components aren't available or if they malfunction, verification might not complete. This failure puts the workload in a degraded state. Some examples of these critical single-point-of-failure dependencies are:

- Ingress and egress firewalls.
- Certificate revocation lists.
- Accurate system time provided by a Network Time Protocol (NTP) server.
- Identity providers, like Microsoft Entra ID.



Tradeoff: Increased complexity of disaster recovery. A workload must reliably recover from all forms of disaster.

- Security controls can extend recovery time objectives. The added time can come from steps needed to decrypt backups or from operational access delays during site reliability triage.
- Security controls themselves, like secret vaults and edge DDoS protection, must be part of the workload's disaster recovery plan and validated in recovery drills.
- Security or compliance requirements might constrain data residency or access controls for backups, which further complicates recovery by segmenting even offline replicas.



Tradeoff: Increased rate of change. A workload that experiences runtime change is exposed to more risk of reliability impact due to that change.

- Stricter patching and update policies lead to more changes in a workload's production environment. This change comes from sources like these:
 - Application code being released more frequently because of updates to libraries or updates to base container images
 - Increased routine patching of operating systems
 - Staying current with versioned applications or data platforms
 - Applying vendor patches to software in the environment
- Rotating keys, service principal credentials, and certificates can cause transient failures during the transition as clients pick up the new value.

Security tradeoffs with Cost Optimization



Tradeoff: Additional infrastructure. One approach to cost optimizing a workload is to look for ways to reduce the diversity and number of components and increase density.

Some workload components and design decisions exist only to protect the security (confidentiality, integrity, and availability) of systems and data. These components enhance security of the environment but increase costs. They must also be subject to cost optimization themselves. Examples of security-centric resources or licensing costs include:

- Compute, network, and data segmentation for isolation, which sometimes involves running separate instances, preventing co-location and reducing density.
- Specialized observability tooling, like a SIEM that can perform aggregation and threat intelligence.
- Specialized networking appliances or capabilities, like firewalls or distributed denial-of-service prevention.
- Data classification tools that are required for capturing sensitivity and information-type labels.
- Specialized storage or compute capabilities to support encryption at rest and in transit, like an HSM or confidential-compute functionality.
- Dedicated testing environments and testing tools to validate that security controls are functioning and to uncover previously undiscovered gaps in coverage.

The preceding items often also exist outside of production environments, in preproduction and disaster recovery resources.



Tradeoff: Increased demand on infrastructure. The Cost Optimization pillar prioritizes driving down demand on resources to enable the use of cheaper SKUs, fewer instances, or reduced consumption.

- *Premium SKUs:* Some security measures in cloud and vendor services that can benefit the security posture of a workload might only be found in more expensive SKUs or tiers.
- *Log storage:* High fidelity security monitoring and audit data that provide broad coverage increase storage costs. Security observability data is also typically retained longer than operational insights.
- *Increased resource consumption:* In-process and on-host security controls add demand for competing resources such as CPU cycles, memory, disk I/O, and network bandwidth.

Encryption in transit and at rest adds demand too. Both can require higher instance counts or larger SKUs.



Tradeoff: Increased process and operational costs. Personnel process costs are part of the overall total cost of ownership and are factored into a workload's return on investment. Optimizing these costs is a recommendation of the Cost Optimization pillar.

- A comprehensive and strict patch management regime increases the time and money spent on routine tasks, and it usually requires investment in preparedness for ad hoc zero-day patching.
- Stricter access controls reduce the risk of unauthorized access but add complexity to user management and operational access.
- Training and awareness for security tools and processes consume employee time and incur costs for materials, instructors, and training environments.
- Regulatory compliance might require additional investment in audits and compliance reporting.
- Planning and running drills for security incident response preparedness takes time.
- Routine and ad hoc security processes, like key or certificate rotation, take time to design and perform.
- Security validation in the SDLC usually requires specialized tools. Your organization might need to pay for those tools, and remediating findings also takes time.
- Hiring third-party practitioners to perform security testing, including penetration testing, incurs costs.

Security tradeoffs with Operational Excellence



Tradeoff: Complications in observability and serviceability. Operational Excellence requires architectures to be serviceable and observable. The most serviceable architectures are transparent to everyone involved.

- Security benefits from high fidelity logging that supports baseline-deviation alerting and incident response. The volume can crowd out insights targeted at reliability or performance.

- Data masking compliance redacts log segments, or even large amounts of tabular data, to protect confidentiality. The team must evaluate how that observability gap affects alerting and hinders incident response.
- Strong resource segmentation complicates observability. It requires additional cross-service distributed tracing and correlation to capture flow traces. The segmentation also increases the compute and data surface to protect.
- Some security controls impede access by design and can slow workload operators' emergency access during incident response. Incident response plans need extra planning and drills to remain effective.



Tradeoff: Decreased agility and increased complexity. Workload teams measure their velocity so that they can improve the quality, frequency, and efficiency of delivery activities over time. Workload complexity factors into the effort and risk involved in operations.

- Stricter change control and approval policies reduce the risk of introducing security vulnerabilities, but they slow the development and safe deployment of new features. At the same time, the need to ship security updates and patches drives demand for more frequent deployments. Additionally, human-gated approval policies in operational processes can make it more difficult to automate those processes.
- Security testing produces findings that the team must prioritize, which can block planned work.
- Routine, ad hoc, and emergency processes might require audit logging to meet compliance requirements. This logging increases the rigidity of running the processes.
- Workload teams might increase the complexity of identity management activities as the granularity of role definitions and assignments is increased.
- More security-related routine tasks, like certificate management, increase the number of processes to automate.



Tradeoff: Increased coordination efforts. A team that minimizes external points of contact and review can control their operations and timeline more effectively.

- As external compliance requirements from the larger organization or from external entities increase, the complexity of achieving and proving compliance with auditors also increases.

- Security requires specialized skills that workload teams don't typically have. Sourcing those skills from the larger organization or from third parties requires coordinating effort, access, and responsibility.
- Communication plans for responsible breach disclosure are often required by compliance or organizational mandates and must be factored into security coordination.

Security tradeoffs with Performance Efficiency



Tradeoff: Increased latency and overhead. A performant workload reduces latency and overhead.

- Inspection security controls, like firewalls and content safety filters, sit in the flows that they secure. Those flows are therefore subject to additional verification, which adds latency to requests. In highly decoupled architectures, inspections can happen multiple times for a single user or data flow transaction.
- Identity controls require each invocation of a controlled component to be verified explicitly. Verification consumes compute cycles and might require network traversal for authorization.
- Encryption and decryption consume compute cycles, which increases the time and resources consumed by those flows. The increase usually correlates with algorithm complexity and with the generation of high-entropy initialization vectors (IVs).
- More extensive logging consumes more system resources and network bandwidth to stream those logs.
- Resource segmentation frequently introduces network hops in a workload's architecture.



Tradeoff: Increased chance of misconfiguration. Reliably meeting performance targets depends on predictable implementations of the design.

Misconfigured or overextended security controls can hurt performance because of inefficient configuration. Examples of security control configurations that can affect performance include:

- Firewall rule ordering, complexity, and quantity (granularity).

- Failing to exclude key files from file integrity monitors or virus scanners. Neglecting this step can lead to lock contention.
- Web application firewalls performing deep packet inspection for languages or platforms that are irrelevant for the components that are being protected.

Related links

Explore the tradeoffs for the other pillars:

- [Reliability tradeoffs](#)
- [Cost Optimization tradeoffs](#)
- [Operational Excellence tradeoffs](#)
- [Performance Efficiency tradeoffs](#)

Last updated on 04/28/2026